

AI CODING ASSIGNMENT 12.1

VARSHITH REDDY-2503A52L15

Task Description #1 (Sorting – Merge Sort Implementation)

- Task: Use AI to generate a Python program that implements the Merge Sort algorithm.
- Instructions:
 - o Prompt AI to create a function `merge_sort(arr)` that sorts a list in ascending order.
 - o Ask AI to include time complexity and space complexity in the function docstring.
 - o Verify the generated code with test cases.
- Expected Output:
 - o A functional Python script implementing Merge Sort with proper documentation

PROMPT :

```
write a python program that implement merge sort algorithm that sorts a  
list of ascending order include time complexity and scape complexity in  
function docstring verify and genarate code run test casess
```

CODE:

`def merge_sort(arr):` """ Sorts a list of numbers in ascending order using the Merge Sort algorithm.

Time Complexity:

- Best Case: $O(n \log n)$
- Average Case: $O(n \log n)$
- Worst Case: $O(n \log n)$

Space Complexity:

- $O(n)$ due to the temporary arrays used for merging.

Args:

`arr (list)`: List of comparable elements (e.g., integers, floats).

Returns:

`list`: A new sorted list in ascending order.

Example:

```
>>> merge_sort([3, 1, 4, 1, 5, 9, 2, 6])  
[1, 1, 2, 3, 4, 5, 6, 9]  
"""
```

```
if len(arr) <= 1:
```

```
    return arr
```

```
mid = len(arr) // 2
```

```
left = merge_sort(arr[:mid])
```

```
right = merge_sort(arr[mid:])
```

```
return merge(left, right)
```

```
def merge(left, right):  
    result = []  
    i = j = 0 # Merge the two sorted  
    lists while i < len(left) and j < len(right):  
        if left[i] <= right[j]:
```

```
result.append(left[i]) i += 1 else: result.append(right[j]) j += 1 #
Append any remaining elements result.extend(left[i:])
result.extend(right[j:]) return result
```

Test cases

```
if name == "main": # Test 1: Random order arr1 = [38, 27, 43, 3, 9,
82, 10] print("Original:", arr1) print("Sorted: ", merge_sort(arr1))
assert merge_sort(arr1) == sorted(arr1)
```

```
# Test 2: Already sorted
```

```
arr2 = [1, 2, 3, 4, 5]
```

```
assert merge_sort(arr2) == [1, 2, 3, 4, 5]
```

```
# Test 3: Reverse order
```

```
arr3 = [5, 4, 3, 2, 1]
```

```
assert merge_sort(arr3) == [1, 2, 3, 4, 5]
```

```
# Test 4: Duplicates
```

```
arr4 = [2, 3, 2, 1, 4, 1]
```

```
assert merge_sort(arr4) == [1, 1, 2, 2, 3, 4]
```

```
# Test 5: Single element
```

```
arr5 = [42]
```

```
assert merge_sort(arr5) == [42]
```

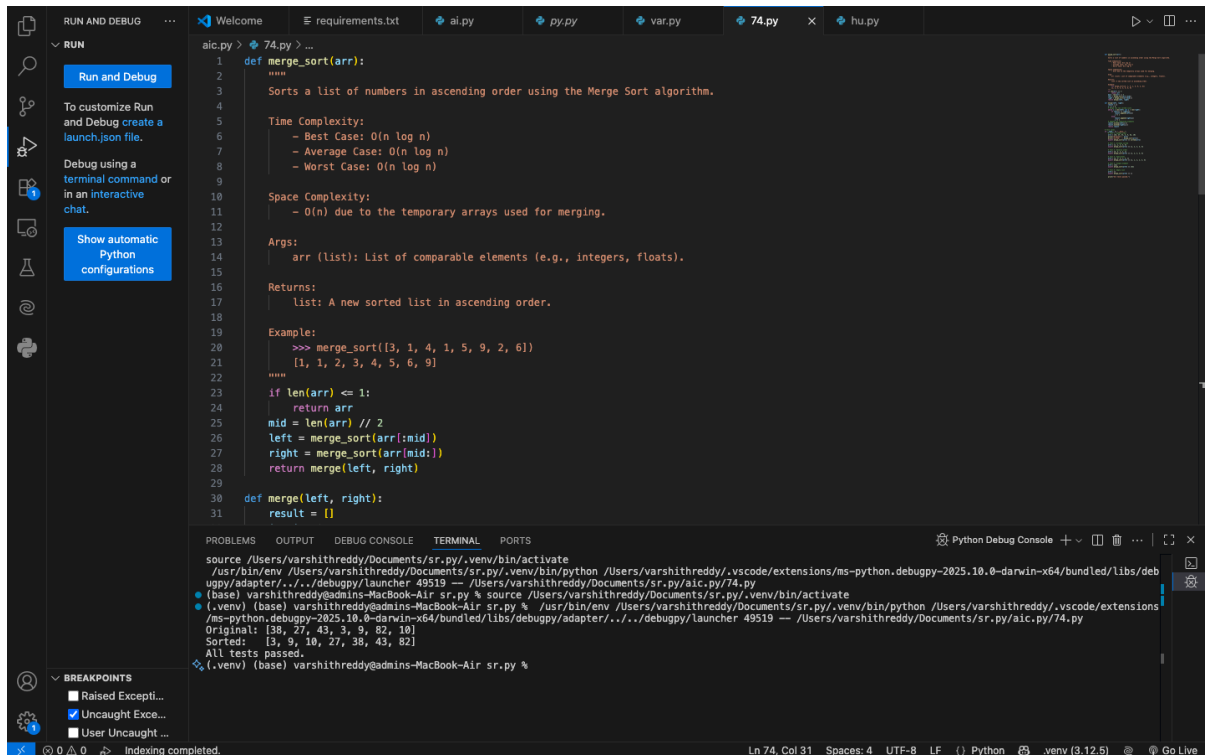
```
# Test 6: Empty list
```

```
arr6 = []
```

```
assert merge_sort(arr6) == []
```

```
print("All tests passed.")
```

OUTPUT



The screenshot shows a VS Code editor with a Python file named `74.py` containing a `merge_sort` function. The function includes docstrings for Time Complexity (Best, Average, and Worst cases), Space Complexity, Args, Returns, and an Example. The code implements a recursive merge sort algorithm. Below the editor, the `TERMINAL` panel shows the execution output, which includes the command to run the script, the original and sorted arrays, and a confirmation that all tests passed.

```
1 def merge_sort(arr):
2     """
3     Sorts a list of numbers in ascending order using the Merge Sort algorithm.
4
5     Time Complexity:
6     - Best Case: O(n log n)
7     - Average Case: O(n log n)
8     - Worst Case: O(n log n)
9
10    Space Complexity:
11    - O(n) due to the temporary arrays used for merging.
12
13    Args:
14    arr (list): List of comparable elements (e.g., integers, floats).
15
16    Returns:
17    list: A new sorted list in ascending order.
18
19    Example:
20    >>> merge_sort([3, 1, 4, 1, 5, 9, 2, 6])
21    [1, 1, 2, 3, 4, 5, 6, 9]
22    """
23    if len(arr) <= 1:
24        return arr
25    mid = len(arr) // 2
26    left = merge_sort(arr[:mid])
27    right = merge_sort(arr[mid:])
28    return merge(left, right)
29
30 def merge(left, right):
31     result = []
```

```
source /Users/varshithreddy/Documents/sr.py/.venv/bin/activate
/usr/bin/env /Users/varshithreddy/Documents/sr.py/.venv/bin/python /Users/varshithreddy/.vscode/extensions/ms-python.debugpy-2025.10.0-darwin-x64/bundled/libs/deb
ugpy/adapter/.../launcher 49519 -- /Users/varshithreddy/Documents/sr.py/aic.py/74.py
(base) varshithreddy@admins-MacBook-Air sr.py % source /Users/varshithreddy/Documents/sr.py/.venv/bin/activate
(.venv) (base) varshithreddy@admins-MacBook-Air sr.py % /usr/bin/env /Users/varshithreddy/Documents/sr.py/.venv/bin/python /Users/varshithreddy/.vscode/extensions
/ms-python.debugpy-2025.10.0-darwin-x64/bundled/libs/debugpy/adapter/.../launcher 49519 -- /Users/varshithreddy/Documents/sr.py/aic.py/74.py
Original: [38, 27, 43, 3, 9, 82, 10]
Sorted: [3, 9, 10, 27, 38, 43, 82]
All tests passed.
(.venv) (base) varshithreddy@admins-MacBook-Air sr.py %
```

Task Description #2 (Searching – Binary Search with AI Optimization)

- Task: Use AI to create a binary search function that finds a target element in a sorted list.
- Instructions:
 - o Prompt AI to create a function `binary_search(arr, target)` returning the index of the target or -1 if not found.
 - o Include docstrings explaining best, average, and worst-case complexities.

- o Test with various inputs.
- Expected Output:
- o Python code implementing binary search with AI-generated comments and docstrings

PROMPT:

WRITE A python code for binary search that search element form sorted list and returning index of the target or -1 if not found include docstring that explaing best ,avg and worst case of complexity and test with various inputs

CODE:

```
def binary_search(arr, target):  
    """  
    Perform binary search on a sorted list to find the index of the target element.  
  
    Args:  
    arr (list): A sorted list of elements.  
    target: The element to search for.  
  
    Returns:  
    int: Index of the target element if found; otherwise, -1.  
  
    Time Complexity:  
    - Best Case:  $O(1)$  - when the target is at the middle position in the first comparison.  
    - Average Case:  $O(\log n)$  - logarithmic time as the list is divided in half at each step.  
    - Worst Case:  $O(\log n)$  - occurs when the target is not present or found at the end of the search.  
  
    Space Complexity:  
    -  $O(1)$  - iterative approach uses constant extra space.  
    """  
    left, right = 0, len(arr) - 1  
  
    while left <= right:  
        mid = left + (right - left) // 2  
        # Debug: print(f'Left: {left}, Right: {right}, Mid: {mid}, Value: {arr[mid]}')  
        if arr[mid] == target:  
            return mid # Target found  
        elif arr[mid] < target:  
            left = mid + 1  
        else:  
            right = mid - 1  
    return -1 # Target not found
```

```
# Tests for various scenarios
if __name__ == "__main__":
    # Test 1: Target is present in the middle
    arr1 = [1, 3, 5, 7, 9, 11]
    print(binary_search(arr1, 7)) # Output: 3

    # Test 2: Target is the first element
    print(binary_search(arr1, 1)) # Output: 0

    # Test 3: Target is the last element
    print(binary_search(arr1, 11)) # Output: 5

    # Test 4: Target is not present
    print(binary_search(arr1, 8)) # Output: -1

    # Test 5: Empty list
    print(binary_search([], 10)) # Output: -1

    # Test 6: Single element, target present
    print(binary_search([42], 42)) # Output: 0

    # Test 7: Single element, target not present
    print(binary_search([42], 99)) # Output: -1

    # Test 8: Large list, target present
    arr2 = list(range(1000000))
    print(binary_search(arr2, 123456)) # Output: 123456

    # Test 9: Large list, target not present
    print(binary_search(arr2, 1000001)) # Output: -1
```

Output:

```
1 def binary_search(arr, target):
2     """
3     Perform binary search on a sorted list to find the index of the target element.
4
5     Args:
6     ... arr (list): A sorted list of elements.
7     ... target: The element to search for.
8
9     Returns:
10    ... int: Index of the target element if found; otherwise, -1.
11
12    Time Complexity:
13    ... Best Case: O(1) -- when the target is at the middle position in the first comparison.
14    ... Average Case: O(log n) -- logarithmic time as the list is divided in half at each step.
15    ... Worst Case: O(log n) -- occurs when the target is not present or found at the end of the search.
16
17    Space Complexity:
18    ... O(1) -- iterative approach uses constant extra space.
19
20    """
21    left, right = 0, len(arr) - 1
22
23    while left <= right:
24        mid = left + (right - left) // 2
25        # debug print(f'Left: {left}, Right: {right}, Mid: {mid}, Value: {arr[mid]}')
26        if arr[mid] == target:
27            return mid # Target found
28        elif arr[mid] < target:
29            left = mid + 1
30        else:
31            right = mid - 1
32    return -1 # Target not found
```

Task Description #3 (Real-Time Application – Inventory Management System)

Scenario: A retail store's inventory system contains thousands of products, each with attributes like product ID, name, price, and stock quantity. Store staff need to:

1. Quickly search for a product by ID or name.
2. Sort products by price or quantity for stock analysis.

- Task:

- Use AI to suggest the most efficient search and sort algorithms for this use case.
- Implement the recommended algorithms in Python.
- Justify the choice based on dataset size, update frequency, and performance requirements.

- Expected Output:

- o A table mapping operation → recommended algorithm → justification.
- o Working Python functions for searching and sorting the inventory.

PROMPT:

A retail store's inventory system contains thousands of products, each with attributes like product ID, name, price, and stock quantity. Store staff need to: 1. Quickly search for a product by ID or name. 2. Sort products by price or quantity for stock analysis. • Task: o Use AI to suggest the most efficient search and sort algorithms for this use case. o Implement the recommended algorithms in Python. o Justify the choice based on dataset size, update frequency, and performance requirements.

CODE:

```
from typing import List, Dict

class Product:
    def __init__(self, product_id: str, name: str, price: float, quantity: int):
        self.product_id = product_id
        self.name = name
        self.price = price
        self.quantity = quantity

    def __repr__(self):
        return f"{self.product_id}: {self.name} (${self.price}, Qty: {self.quantity})"

class InventorySystem:
    def __init__(self):
        self.products: List[Product] = []
        self.product_by_id: Dict[str, Product] = {}

    def add_product(self, product: Product):
        self.products.append(product)
        self.product_by_id[product.product_id] = product

    def search_by_id(self, product_id: str) -> Product:
        return self.product_by_id.get(product_id)

    def search_by_name(self, name: str) -> List[Product]:
        # Case-insensitive partial match
        return [p for p in self.products if name.lower() in p.name.lower()]
```



```
def sort_by_price(self, reverse: bool = False) -> List[Product]:
    return sorted(self.products, key=lambda x: x.price, reverse=reverse)

def sort_by_quantity(self, reverse: bool = False) -> List[Product]:
    return sorted(self.products, key=lambda x: x.quantity, reverse=reverse)

# Example usage
if __name__ == "__main__":
    inv = InventorySystem()
    inv.add_product(Product("P001", "Apple", 0.99, 50))
    inv.add_product(Product("P002", "Banana", 0.59, 100))
    inv.add_product(Product("P003", "Orange", 1.29, 80))

# Search by ID
print("Search by ID 'P002':", inv.search_by_id("P002"))

# Search by Name
print("Search by name 'an':", inv.search_by_name("an"))

# Sort by Price
print("Sort by price:", inv.sort_by_price())

# Sort by Quantity
print("Sort by quantity (descending):", inv.sort_by_quantity(reverse=True))
```

OUTPUT:

RUN AND DEBUG

Run and Debug

To customize Run and Debug create a launch.json file.

Debug using a terminal command or in an interactive chat.

Show automatic Python configurations

74.py

InventorySystem

__init__

1 from typing import List, Dict

2

3 class Product:

4 def __init__(self, product_id: str, name: str, price: float, quantity: int):

5 self.product_id = product_id

6 self.name = name

7 self.price = price

8 self.quantity = quantity

9

10 def __repr__(self):

11 return f"({self.product_id}: {self.name} ({self.price}, Qty: {self.quantity}))"

12

13 class InventorySystem:

14 def __init__(self):

15 self.products: List[Product] = []

16 self.product_by_id: Dict[str, Product] = {}

17

18 def add_product(self, product: Product):

19 self.products.append(product)

20 self.product_by_id[product.product_id] = product

21

22 def search_by_id(self, product_id: str) -> Product:

23 return self.product_by_id.get(product_id)

24

25 def search_by_name(self, name: str) -> List[Product]:

26 # Case-insensitive partial match

27 return [p for p in self.products if name.lower() in p.name.lower()]

28

29 def sort_by_price(self, reverse: bool = False) -> List[Product]:

30 return sorted(self.products, key=lambda x: x.price, reverse=reverse)

31

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

Python Debug Console

source /Users/varshithreddy/Documents/sr.py/.venv/bin/activate

/usr/bin/env /Users/varshithreddy/Documents/sr.py/.venv/bin/python /Users/varshithreddy/.vscode/extensions/ms-python.debugpy-2025.10.0-darwin-x64/bundled/libs/debugpy/adapter/../../debugpy/launcher 49945 -- /Users/varshithreddy/Documents/sr.py/aic.py/74.py

(base) varshithreddy@admins-MacBook-Air sr.py % source /Users/varshithreddy/Documents/sr.py/.venv/bin/activate

(.venv) (base) varshithreddy@admins-MacBook-Air sr.py % /usr/bin/env /Users/varshithreddy/Documents/sr.py/.venv/bin/python /Users/varshithreddy/.vscode/extensions/ms-python.debugpy-2025.10.0-darwin-x64/bundled/libs/debugpy/adapter/../../debugpy/launcher 49945 -- /Users/varshithreddy/Documents/sr.py/aic.py/74.py

Search by ID "P002": P002: Banana (\$0.59, Qty: 100)

Search by name "an": [P002: Banana (\$0.59, Qty: 100), P003: Orange (\$1.29, Qty: 80)]

Sort by price: [P002: Banana (\$0.59, Qty: 100), P001: Apple (\$0.99, Qty: 50), P003: Orange (\$1.29, Qty: 80)]

Sort by quantity (descending): [P002: Banana (\$0.59, Qty: 100), P003: Orange (\$1.29, Qty: 80), P001: Apple (\$0.99, Qty: 50)]

(.venv) (base) varshithreddy@admins-MacBook-Air sr.py %

BREAKPOINTS

■ Raised Excepti...

■ Uncaught Exce...

■ User Uncaught ...

Ln 15, Col 42 Spaces: 4 UTF-8 LF Python .venv (3.12.5) Go Live